

Practical 9

Aim: Create a binary search tree (BST) in C++ and perform insert, delete, and search operations.

Source-code:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
};
Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
Node* insert(Node* root, int value) {
    if (root == NULL)
        return createNode(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    return root;
}
bool search(Node* root, int value) {
    if (root == NULL)
        return false;
    if (root->data == value)
        return true;
    else if (value < root->data)
        return search(root->left, value);
    else
        return search(root->right, value);
}
Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}
Node* deleteNode(Node* root, int value) {
    if (root == NULL)
        return root;
    if (value < root->data)
        root->left = deleteNode(root->left,
value);
    else if (value > root->data)
        root->right = deleteNode(root->right,
value);
    else {
        if (root->left == NULL) {
            Node* temp = root->right;
```

```
delete root;
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right,
temp->data);
    }
    return root;
}
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
int main() {
    Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);
    cout << "Inorder Traversal: ";
    inorder(root);
    cout << "\nSearch 40: ";
    if (search(root, 40))
        cout << "Found";
    else
        cout << "Not Found";
    cout << "\nDeleting 20...\n";
    root = deleteNode(root, 20);
    cout << "Inorder after deletion: ";
    inorder(root);
    return 0;
}
```

Output:

Inorder Traversal: 20 30 40 50 60 70 80

Search 40: Found

Deleting 20...

Inorder after deletion: 30 40 50 60 70 80